Publicar la aplicación en Render

Ingeniería de Métodos | Tutorial para quien administra el piloto | 9 de noviembre de 2026

1. Antes de comenzar

Necesitas una cuenta de GitHub con el repositorio de la aplicación y una cuenta de Render. Usa la estructura de **01-guia-github.pdf**: backend/ y frontend/ directamente en la raíz del repositorio.

La aplicación actual es FastAPI, no Flask. El comando de este tutorial coincide con backend/main.py. El servidor ya entrega tanto las páginas como los cálculos, por lo que basta un Web Service. No necesitas contratar un servicio separado para la interfaz.

El piloto actual no necesita PostgreSQL ni SECRET_KEY. Añadir esas variables no implementa cuentas ni guardado en la nube. Los pasos opcionales de la sección 6 explican cómo configurarlas cuando exista código que las utilice.

2. Crear tu cuenta y conectar GitHub

1. Abre Render y selecciona la opción de crear una cuenta si todavía no tienes una. Puedes usar GitHub como método de acceso o el registro disponible por correo.
2. Completa personalmente el acceso, la verificación de correo y la aceptación de condiciones que Render solicite.
3. En el panel, abre **Account Settings**. Busca **Account Security > Git Deployment Credentials** y selecciona **Add credential > GitHub**. Si el repositorio ya aparece al crear un servicio, la conexión puede estar realizada.
4. En GitHub, revisa los permisos de la integración. Cuando permita elegir repositorios, concede acceso únicamente a **ingenieria-metodos-app**.
5. Regresa a Render. Si el repositorio no aparece, revisa la cuenta u organización elegida y el acceso concedido a la aplicación de Render.

Referencia: conexión del proveedor Git.

3. Crear el servicio web

En el panel, selecciona **New > Web Service**, elige **Git Provider** y conecta el repositorio. Render te mostrará un formulario. Completa los valores siguientes:

Campo	Valor para esta entrega
Name	ingenieria-metodos-piloto, o un nombre disponible
Language / Runtime	Python 3
Branch	main, o la rama donde subiste la versión del piloto
Region	Oregon; conserva la misma región si añades PostgreSQL después
Root Directory	Déjalo vacío: backend/ y frontend/ están en la raíz
Build Command	python -m pip install -r backend/requirements.txt
Start Command	Usa el comando completo del recuadro siguiente
Instance / Compute plan	Free para iniciar las pruebas sin contratar una instancia
Health Check Path	/api/opciones
Auto-Deploy	On Commit, si quieres publicar cada cambio de la rama conectada

Start Command: copia esta línea completa.

```
python -m uvicorn main:app --app-dir backend --host 0.0.0.0 --port $PORT
```

main:app identifica el objeto app de backend/main.py. --app-dir backend permite importar los archivos Python hermanos sin cambiar la raíz del servicio. El host 0.0.0.0 acepta las conexiones de Render y $PORT usa el puerto que la plataforma proporciona. No añadas --reload: es una opción de desarrollo.

Si tu repositorio contiene una carpeta adicional llamada ingenieria-metodos-app, coloca **ingenieria-metodos-app** en Root Directory y conserva los comandos anteriores. No pongas solo backend: el servicio necesita también la carpeta frontend hermana.

Render leerá **.python-version** para seleccionar Python 3.14.7. Si ya existe una variable PYTHON_VERSION con otro valor en el panel, corrígela o elimínala: tiene prioridad sobre el archivo. Referencias: FastAPI en Render, versión de Python.

4. Publicar y obtener el enlace

1. Comprueba que el plan elegido sea Free y que los comandos coincidan con los de esta guía.
2. Pulsa **Create Web Service**. Render descargará el repositorio, instalará las dependencias y arrancará la aplicación.
3. Sigue el progreso en **Deploys / Events** y consulta los registros si aparece un error. Espera a que el despliegue se marque como exitoso o Live.
4. Copia la dirección HTTPS del servicio que muestra Render. Tendrá un dominio **onrender.com**. El nombre exacto lo asigna la plataforma; no existe un enlace público hasta que completes el despliegue.

5. Abre esa dirección en otra ventana y verifica que aparece la portada, que puedes entrar a los módulos y que los cálculos responden.
6. Comparte el enlace de la aplicación, no el enlace al panel de administración ni el del repositorio. Los participantes solo necesitan navegador e internet.

Referencia: servicios web de Render.

5. Alternativa con el archivo render.yaml

La entrega incluye **render.yaml** con el mismo comando de instalación, el comando de inicio, la región, el plan gratuito y la comprobación /api/opciones. Si prefieres usarlo, crea un **Blueprint** desde el panel y selecciona el repositorio y la rama que contienen ese archivo en la raíz. Revisa los recursos propuestos antes de aplicarlo.

El Blueprint crea un único servicio web y no crea bases de datos. Usa esta ruta o la creación manual de la sección 3; no necesitas crear dos servicios para la misma aplicación. Referencia: Blueprint de Render.

6. Variables de entorno: SECRET_KEY y DATABASE_URL

Situación actual: el código no lee estas variables y no tiene modelos ni migraciones de base de datos. Puedes desplegar el piloto sin ellas. No es necesario crear una base para que funcionen los cálculos y los archivos de estudio.

Cómo añadir cualquier variable

Abre tu servicio web en Render, entra a **Environment** y pulsa **Add Environment Variable**. Introduce el nombre en **Key** y el valor real en **Value**, sin añadir comillas externas. Selecciona **Save and deploy** para que el servicio existente reciba los cambios; si cambiaste dependencias o la configuración de instalación, usa **Save, rebuild, and deploy**.

Si eliges **Save only**, el proceso seguirá con sus valores anteriores hasta el próximo despliegue. Nunca escribas secretos dentro de render.yaml, archivos JavaScript o GitHub. Referencia: variables y secretos.

SECRET_KEY, cuando se implementen sesiones o autenticación

Genera un valor aleatorio largo en tu propio equipo y guárdalo como **SECRET_KEY** en Environment. Este comando genera un valor; ejecútalo tú y conserva su salida de forma privada:

```
python -c "import secrets; print(secrets.token_urlsafe(48))"
```

La futura función que firme sesiones o tokens deberá leer esa variable. No la regeneres en cada arranque. Cambiarla puede invalidar sesiones existentes. En esta versión, configurarla no añade una pantalla de acceso ni restringe el enlace del piloto.

DATABASE_URL, cuando se implemente PostgreSQL

1. En Render, selecciona **New > Postgres**. Asigna un nombre y elige la misma región y cuenta que el servicio web.
2. Revisa el plan y crea la base. Espera a que esté disponible.
3. En la página de la base, abre **Connect** y localiza la **Internal Database URL**. Esa conexión sirve para tu aplicación en Render; la externa se reserva para conexiones desde fuera.
4. Copia la URL completa a una variable **DATABASE_URL** del servicio web. Trátala como una contraseña: incluye credenciales.

5. Cuando el código ya tenga un cliente PostgreSQL, modelos, migraciones y operaciones de guardado, instala sus dependencias y ejecuta las migraciones según esa implementación.
6. Verifica guardar y recuperar un registro mediante el servidor. Definir DATABASE_URL por sí solo no realiza esa prueba ni cambia el guardado actual.

Referencia: crear y conectar PostgreSQL.

Aclaración sobre Flask y Gunicorn

En una aplicación diferente que realmente use Flask y tenga app = Flask(...) dentro de app.py, un comando habitual sería **gunicorn app:app**, añadiendo Gunicorn a sus dependencias. **No copies ese comando en este proyecto FastAPI.** La configuración preparada usa Uvicorn y no requiere Procfile ni Gunicorn. Referencia: Flask en Render.

7. Comprobación antes de invitar participantes

1. En Ishikawa, introduce un problema ficticio, agrega una causa y descarga el JPG.
2. En Hombre-Máquina, selecciona Servicio sincrónico y prueba 3 máquinas, L = 1 min, M = 4 min y W = 0.2 min. Debe resultar un ciclo de 5 min y 36 piezas/h.
3. En Balanceo, calcula con las cuatro tareas iniciales y ciclo de 2 min. Debe proponer 2 estaciones, con eficiencia de 82.5 %.
4. Guarda un archivo de cada módulo. Recarga la página y comprueba el borrador. Después usa Cargar estudio para recuperar el archivo del mismo módulo.
5. Comprueba los tres módulos en los dispositivos del piloto. En pantallas pequeñas, revisa también el desplazamiento de tablas y diagramas.
6. Pide a otra persona que abra el enlace desde su propio equipo. Sus borradores deben ser independientes de los tuyos. El responsable no recibe automáticamente sus estudios: se entregan como archivos.

Los borradores creados en localhost no aparecen automáticamente en onrender.com. Antes de cambiar de dirección, descarga los estudios y cárgalos en el enlace nuevo. Conserva la misma dirección durante el piloto.

8. Problemas frecuentes del despliegue

Síntoma	Qué revisar
No encuentra requirements.txt	Build Command debe usar backend/requirements.txt y la raíz debe contener backend/.
Could not import module main	Comprueba --app-dir backend y la existencia de backend/main.py.
No encuentra frontend	No despliegues únicamente backend/. Conserva las dos carpetas hermanas.
No detecta un puerto abierto	Usa --host 0.0.0.0 y --port $PORT.
La portada abre pero no calcula	Comprueba que creaste Web Service, no Static Site, y consulta los registros de la API.
La primera visita tarda	El plan Free se suspende tras 15 minutos sin tráfico y puede tardar cerca de un minuto en despertar.
No aparecen cambios nuevos	Comprueba rama y commit desplegados, o usa Manual Deploy. Recarga el navegador.
SECRET_KEY o DATABASE_URL no cambian el comportamiento	Esta versión no las utiliza; falta implementar las funciones correspondientes.

El sistema de archivos del servidor es temporal. No lo uses para conservar estudios o una base SQLite entre despliegues. El plan gratuito de PostgreSQL tiene vencimiento a los 30 días según las condiciones consultadas; no lo consideres almacenamiento permanente. Para un piloto que necesite disponibilidad inmediata o datos centrales duraderos, revisa un plan adecuado antes de contratarlo. Referencia: límites gratuitos.

9. Estado de esta entrega

Configuración y documentación preparadas localmente. Al comprobar el acceso, GitHub y Render mostraban opciones de iniciar sesión y todavía no existía el repositorio. **No hay despliegue público realizado ni URL de piloto verificada en esta entrega.** La publicación requiere crear el repositorio, iniciar sesión y autorizar su conexión con Render.